

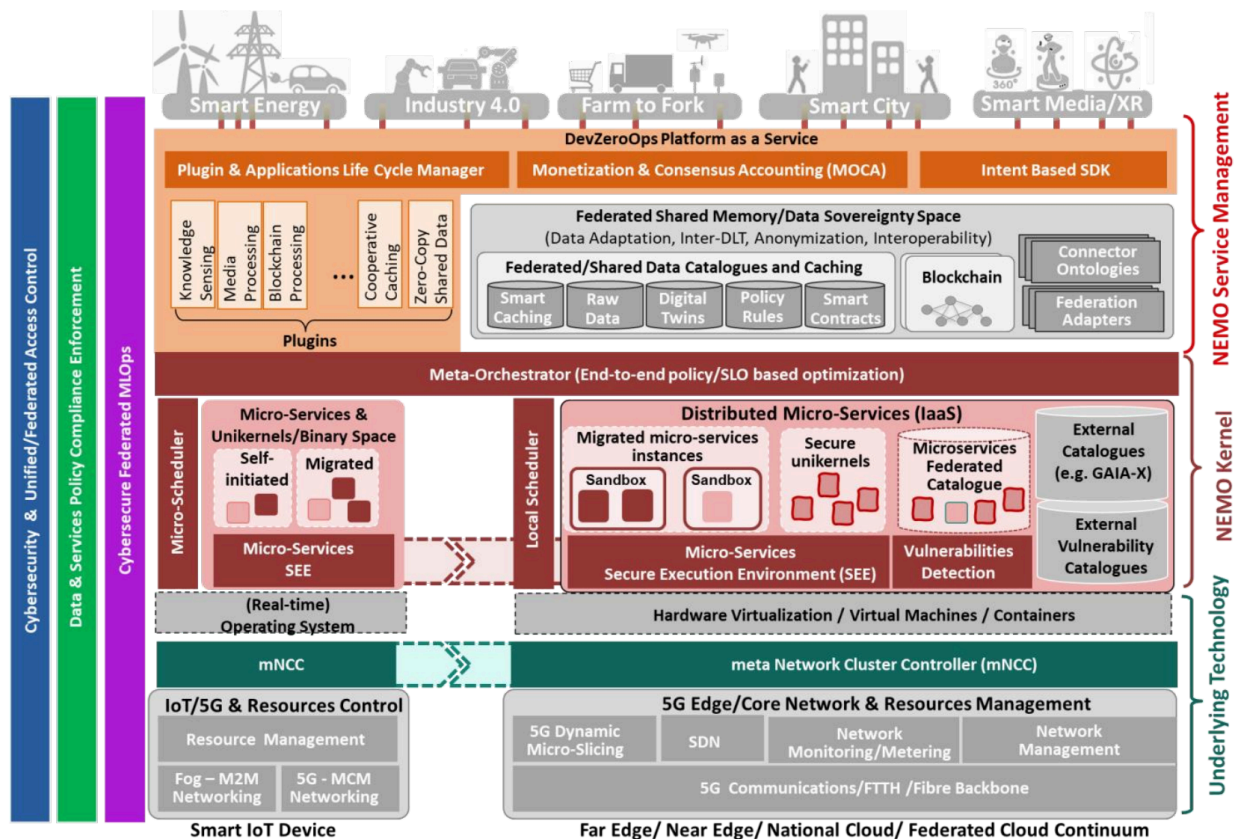
Peer-to-peer protocols for efficient and resilient decentralized learning

Mohamed Amine Legheraba

LIP6 · Sorbonne Université

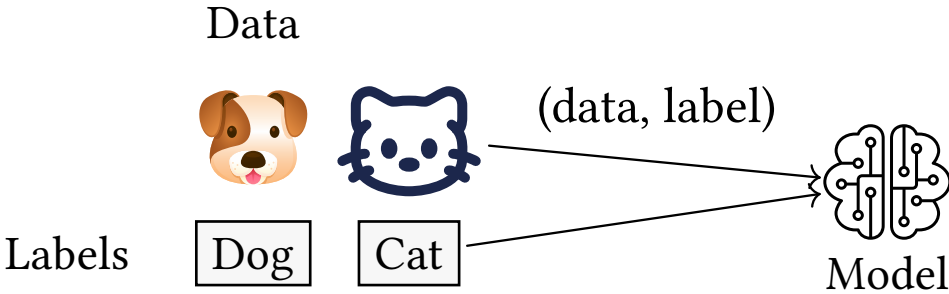
September 7, 2026

Context

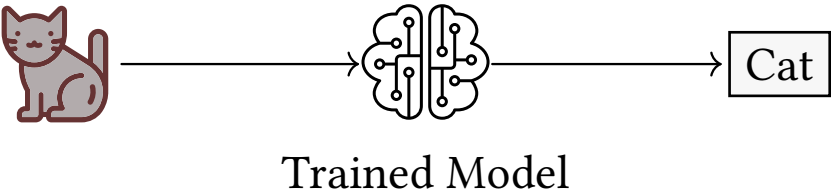


Supervised Classification

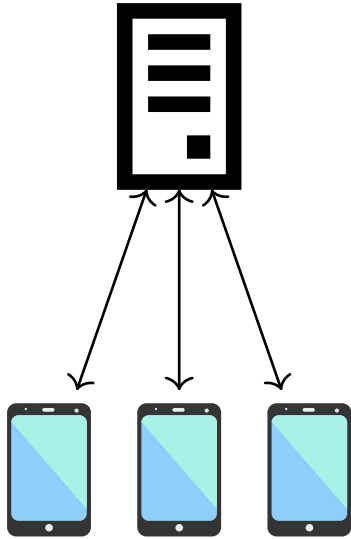
Training



Prediction



Federated Learning*

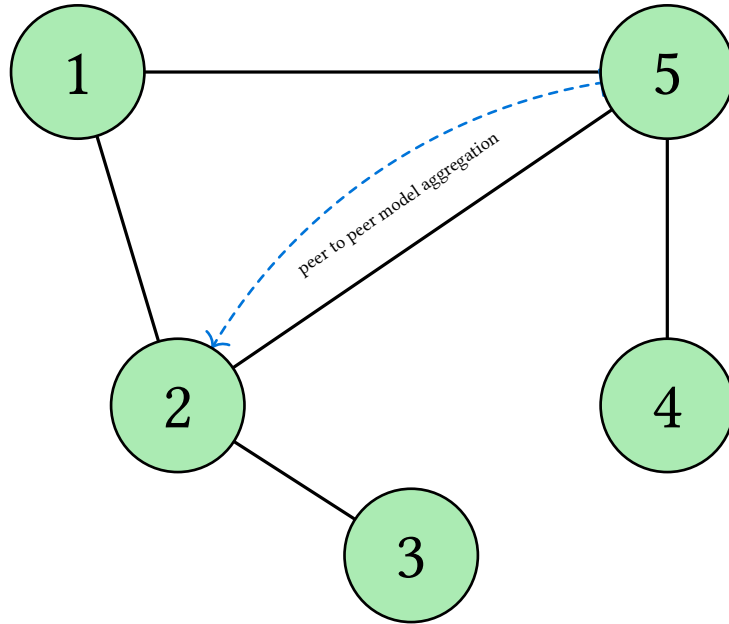


- **Decentralizes the computation load**
- **Keeps data local** : only models are shared

Structured vs Unstructured networks

show differences

Gossip Learning*



Gossip Learning (main thread)

ML model: **model**

List of neighbors : **cache**

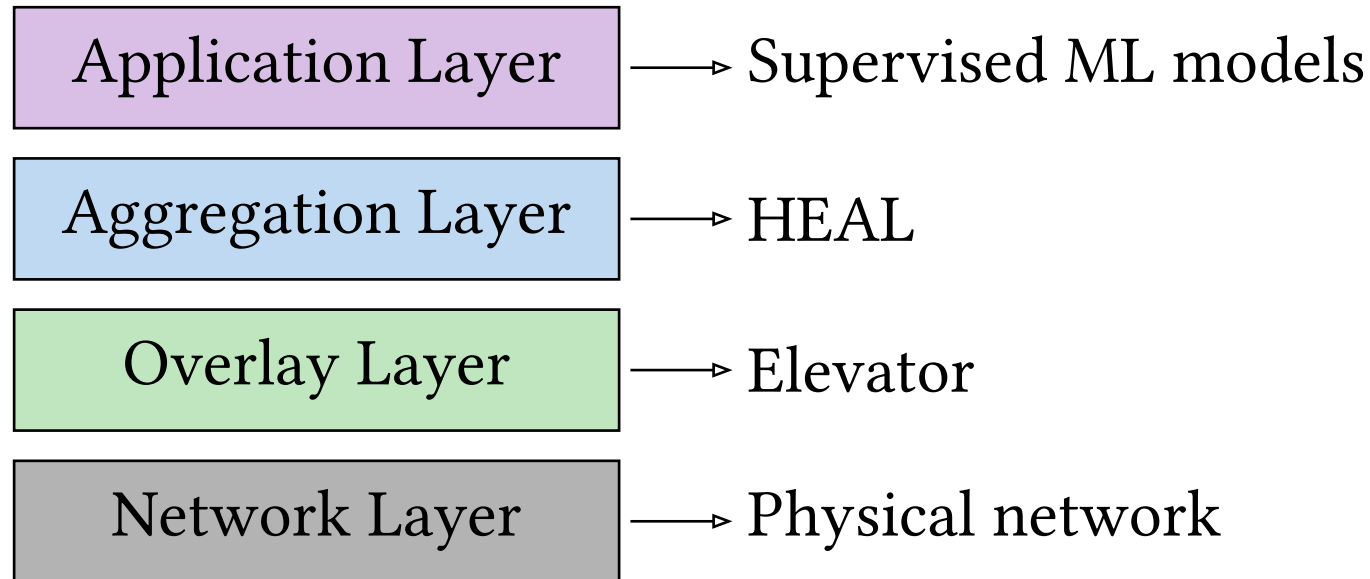
1 **loop**

2 Wait for Δ time units

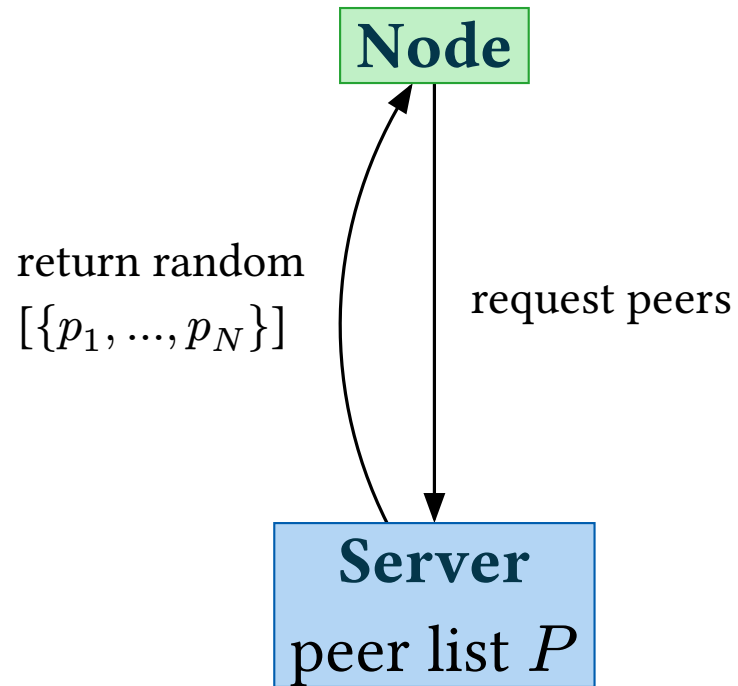
3 peer $\leftarrow$ selectRandom(cache)

4 send(peer, model)

Architecture

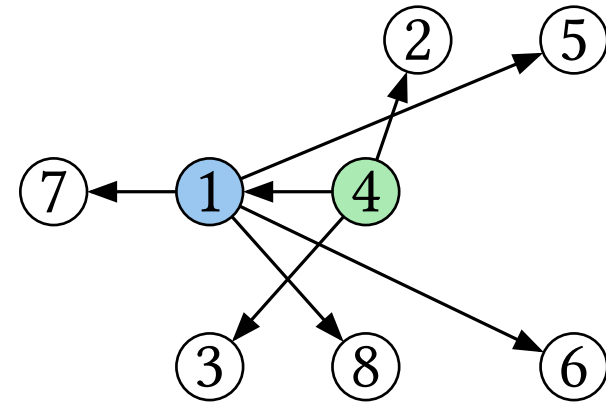
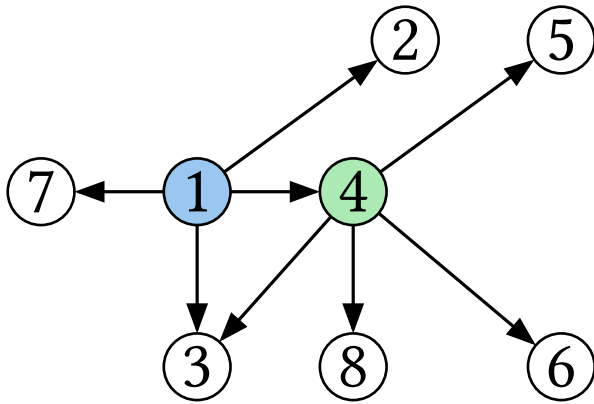


Peer sampling



Definition. (Partial View) The **partial view** of v , denoted $P(v)$, is the set of nodes v can send messages to, $P(v) \subseteq V$, with $|P(v)| \leq c$, $c \ll N$, and $P(v) = \text{neigh}_G(v)$.

Decentralized peer sampling^{*}



Before and after a shuffling operation. Node 1 sends addresses {itself, 2, 3} to node 4. Node 4 sends back {5, 6, 8}.

Metrics

Overlay

- In-degree distribution
- Clustering coefficient
- Average path length
- Diameter
- Convergence time

Machine learning

- Accuracy

Failures

Overlay

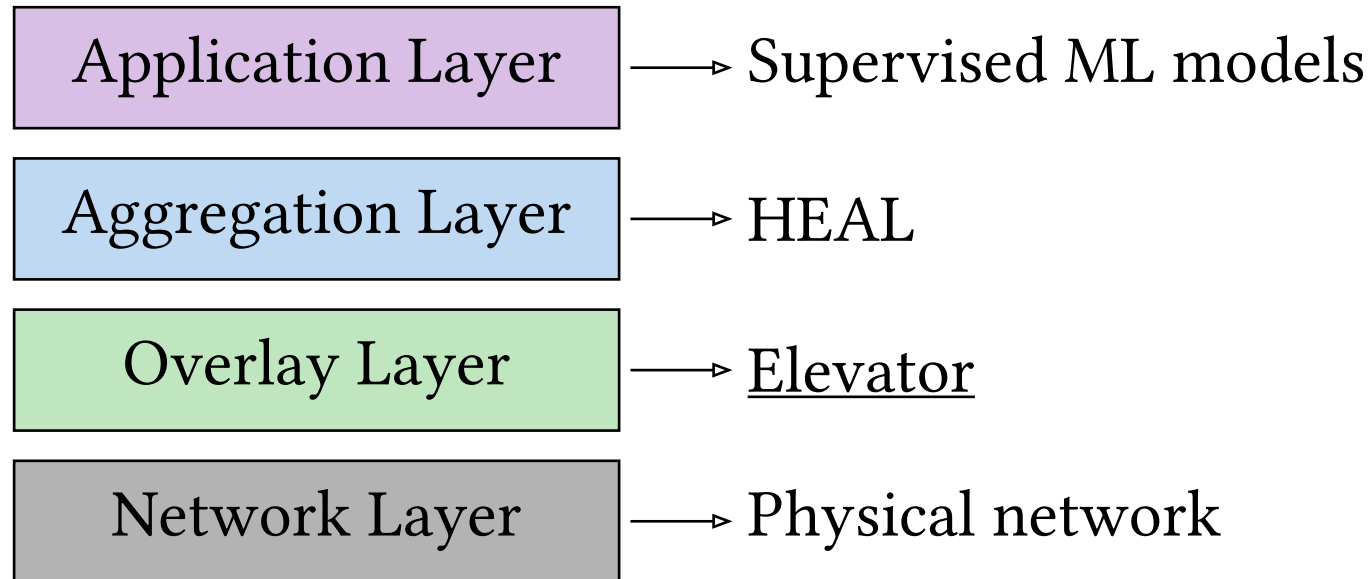
- **Crash failures**
- **Churn** (dynamic joins / leaves)
- **Byzantine failures**

Machine learning

- **Privacy attacks**
- **Poisoning attacks**

Note: ML-level failures are not addressed in this thesis

Architecture



Hub-based topology^{*}

Definition. (Hub) Let $G = (V, E)$ be the directed overlay graph, where each node $v \in V$ keeps a partial view $P(v) \subseteq V$.

A node $h \in V$ is a **hub** if it appears in the partial view of every node:

$$\forall v \in V, \quad h \in P(v)$$

Preferential Attachment

Preferential Attachment — drawing from the concept pioneered by Barabási and Albert^{*}, new connections are established preferentially with nodes that already have many connections.

This enables the organic emergence of hubs: selected nodes naturally assume central roles based on their connectivity, without any explicit distinction other than their number of incoming links.

^{*}[1] A.-L. Barabási, H. Jeong, Z. Néda, E. Ravasz, A. Schubert, and T. Vicsek, “Evolution of the social network of scientific collaborations,” *Physica A: Statistical mechanics and its applications*, vol. 311, no. 3–4, pp. 590–614, 2002.

Random Attachment

Random Attachment — inspired by gossip-based peer sampling algorithms^{**}, nodes maintain a representative and diverse subset of the network.

This prevents excessive clustering and dependency on specific hubs.

When hubs disappear (failures or departure), other nodes are opportunistically elevated to hub status, ensuring continuity and adaptability.

^{*}[2] A. Stavrou, D. Rubenstein, and S. Sahu, “A lightweight, robust p2p system to handle flash crowds,” in *10th IEEE International Conference on Network Protocols, 2002. Proceedings.*, 2002, pp. 226–235.

^{*}[3] M. Jelasity, S. Voulgaris, R. Guerraoui, A.-M. Kermarrec, and M. Van Steen, “Gossip-based peer sampling,” *ACM Transactions on Computer Systems (TOCS)*, vol. 25, no. 3, p. 8–es, 2007.

Elevator in a nutshell

Mechanism

1. Each node asks its **neighbours** for **their** neighbour lists.
2. It connects to the **h most frequent** nodes.
3. It asks these h hubs for extra incoming connections, to fill a **random subset** of its view.

Observed

- After a few cycles, **hubs emerge** (the preferred h are the same for everyone);
- Hubs are **elected at random** among the network nodes;
- Even after **hub failures**, new nodes are elected as hubs.

Example of topology generated

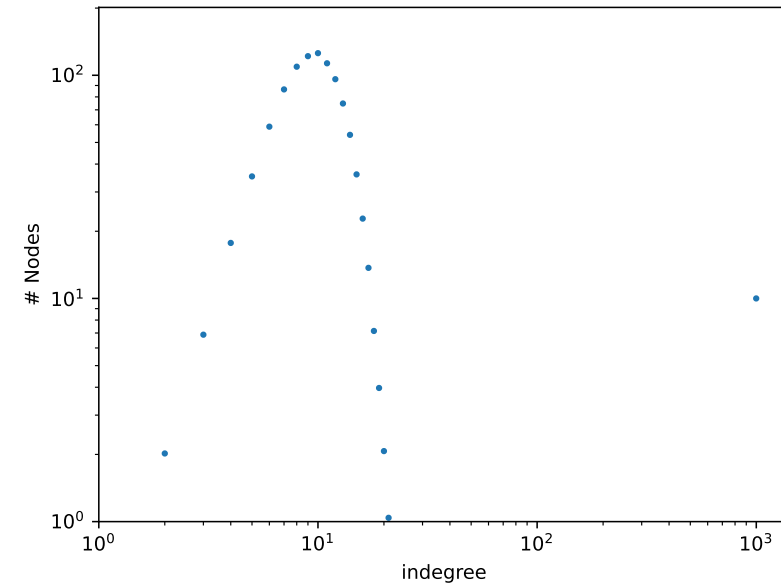
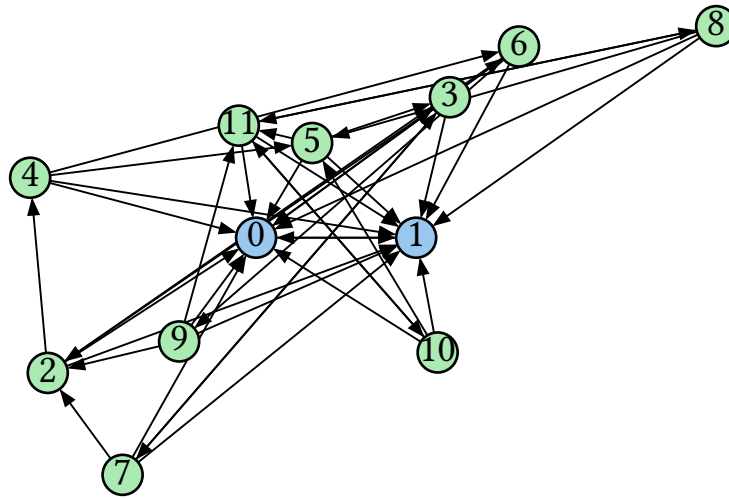


Figure 1: Indegree distribution of a network generated with Elevator, with 1000 nodes and 10 hubs

Mathematical analysis

- **Proof of stability:** once converged, the set of h hubs stays constant (w.h.p.).
- Proof of convergence
- Speed of convergence

Simulations

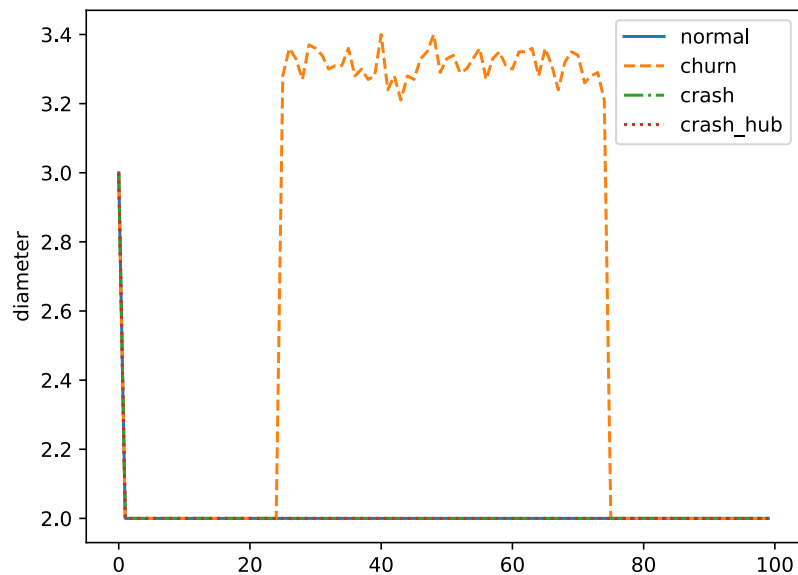
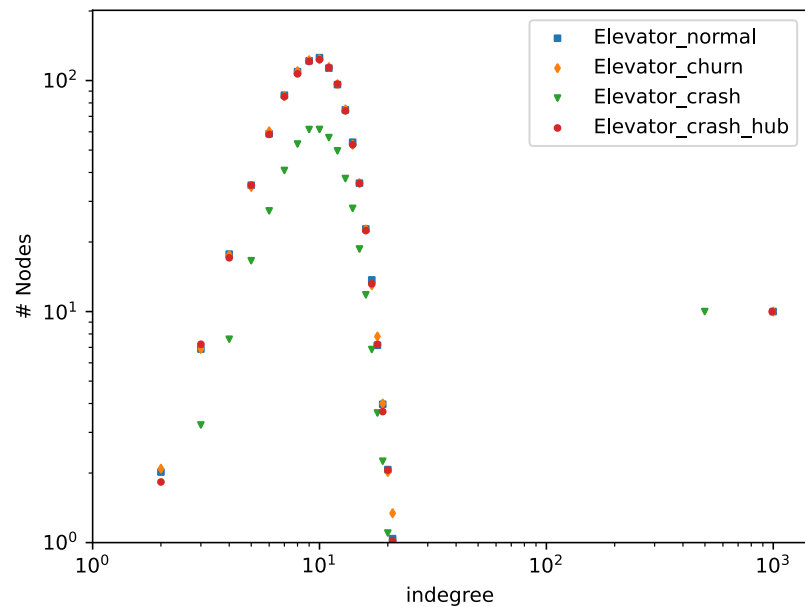
PeerSim was **forked and substantially rewritten** for this thesis:

- Migrated from **SVN to Git**, build rewritten in **Gradle**;
- **Docker + GitLab CI/CD**;
- **Parallelised** the cycle-based engine;
- Failure models from scratch (**crash, churn, Byzantine**);

Parameters

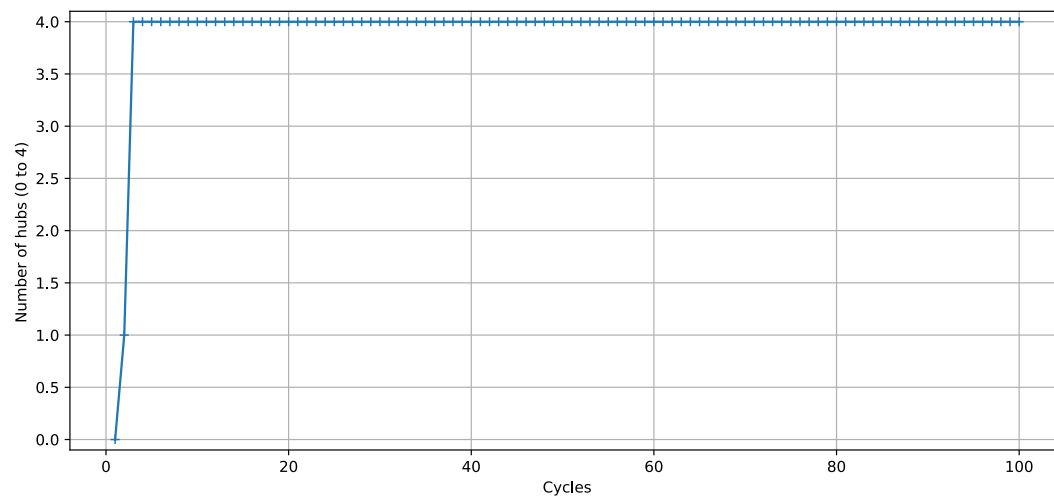
- Network size: **$N = 1000$ nodes**;
- **1000 cycles**, repeated **100 times**;
- Initial topology: **random k -out graph** with **$k = c = 20$** ;
- **$h = 10$ hubs**;
- e.g. brutal crash: **50% of nodes** disconnected at cycle **500**.

Resilience



Real TCP/IP experiments

- **Go + libp2p** over **TCP/IP streams** (+ light **HTTP** per node);
- networks of **20 to 50 nodes** on **1–2 machines**;
- modes: **synchronous / externally-synchronized / asynchronous**;
- **hubs emerge within the first few cycles**
— matching theory & simulation.

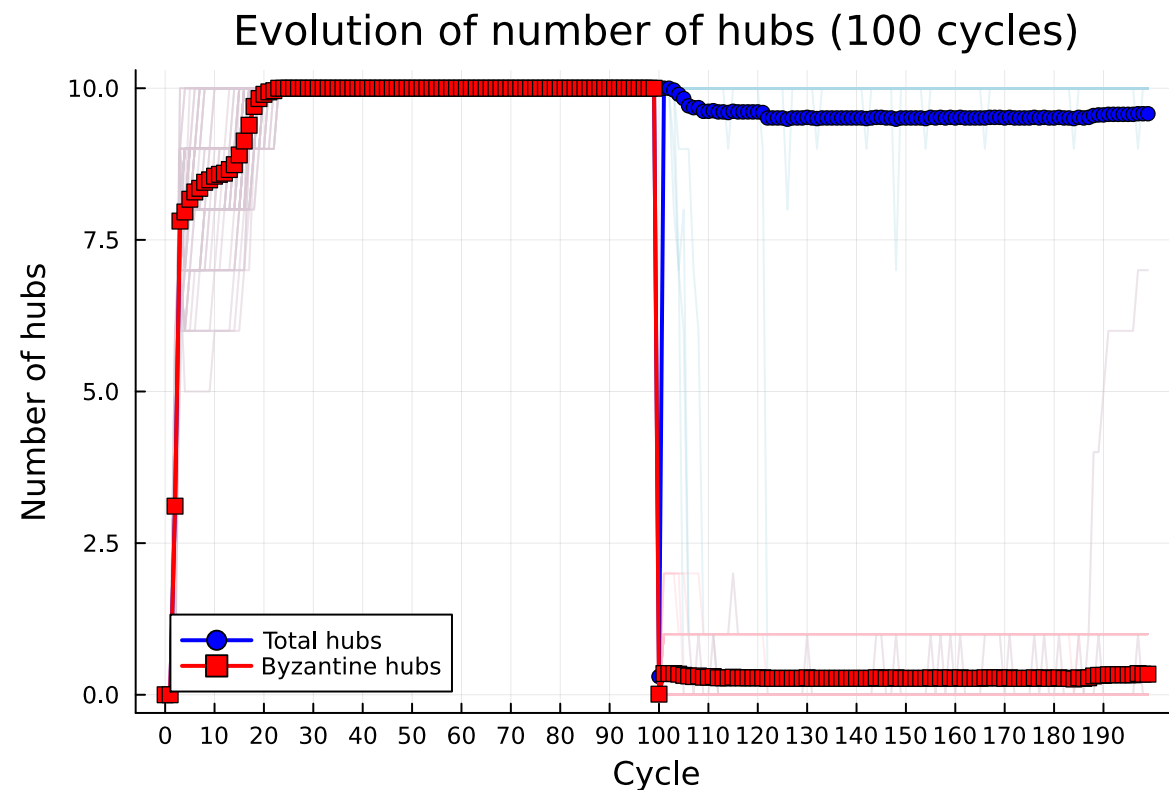


Malicious nodes

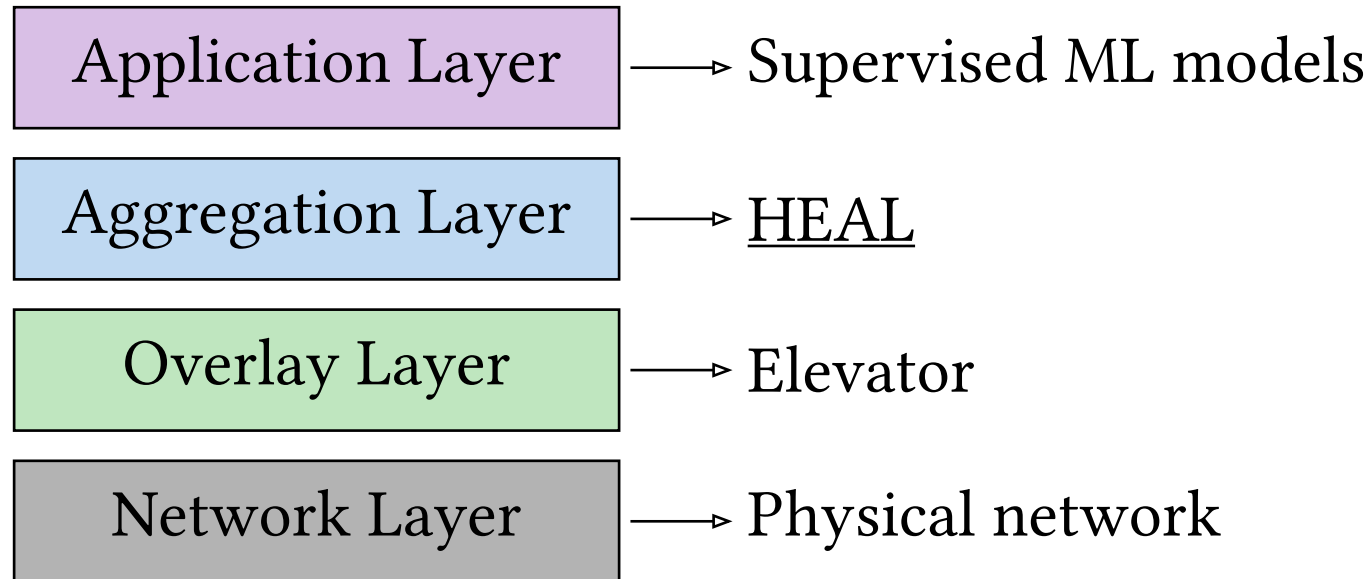
attacks

Present Lift

Resilience against colluding attackers



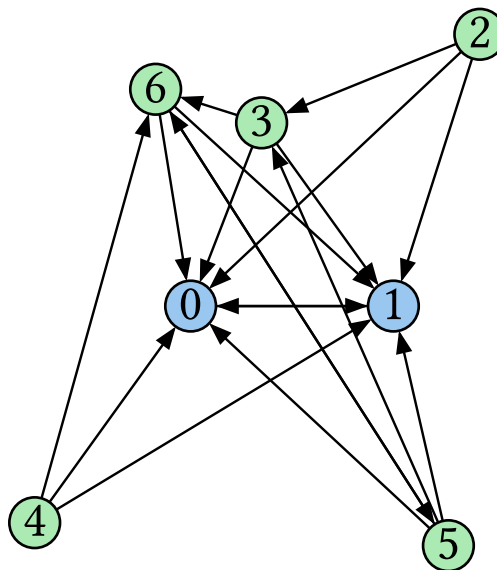
Architecture



Hub Learning Protocol*

1) Each node train a local model

2) Each node sends its model to a (random) hub



3) Hubs receive and aggregate models

4) Hubs compute a global model together

5) Hubs send back the global model to the nodes

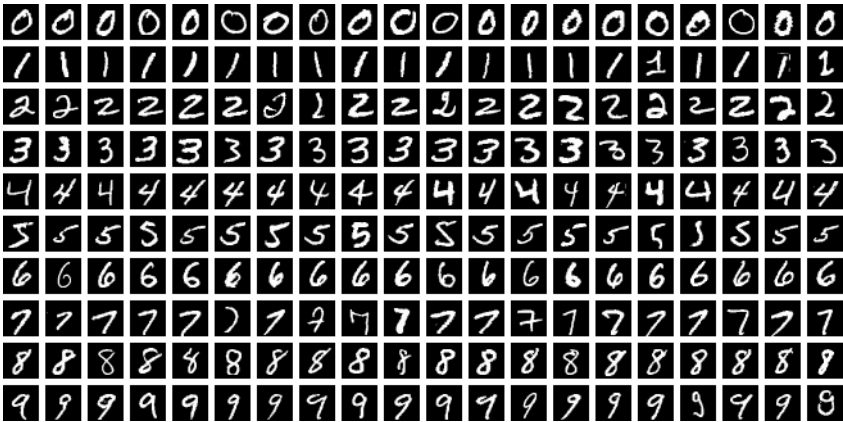
Datasets & Models

Spambase + Logistic regression

Variables Table		
Variable Name	Role	Type
word_freq_make	Feature	Continuous
word_freq_address	Feature	Continuous
word_freq_all	Feature	Continuous
word_freq_3d	Feature	Continuous
word_freq_our	Feature	Continuous
word_freq_over	Feature	Continuous
word_freq_remove	Feature	Continuous

4601 emails, binary
57 parameters

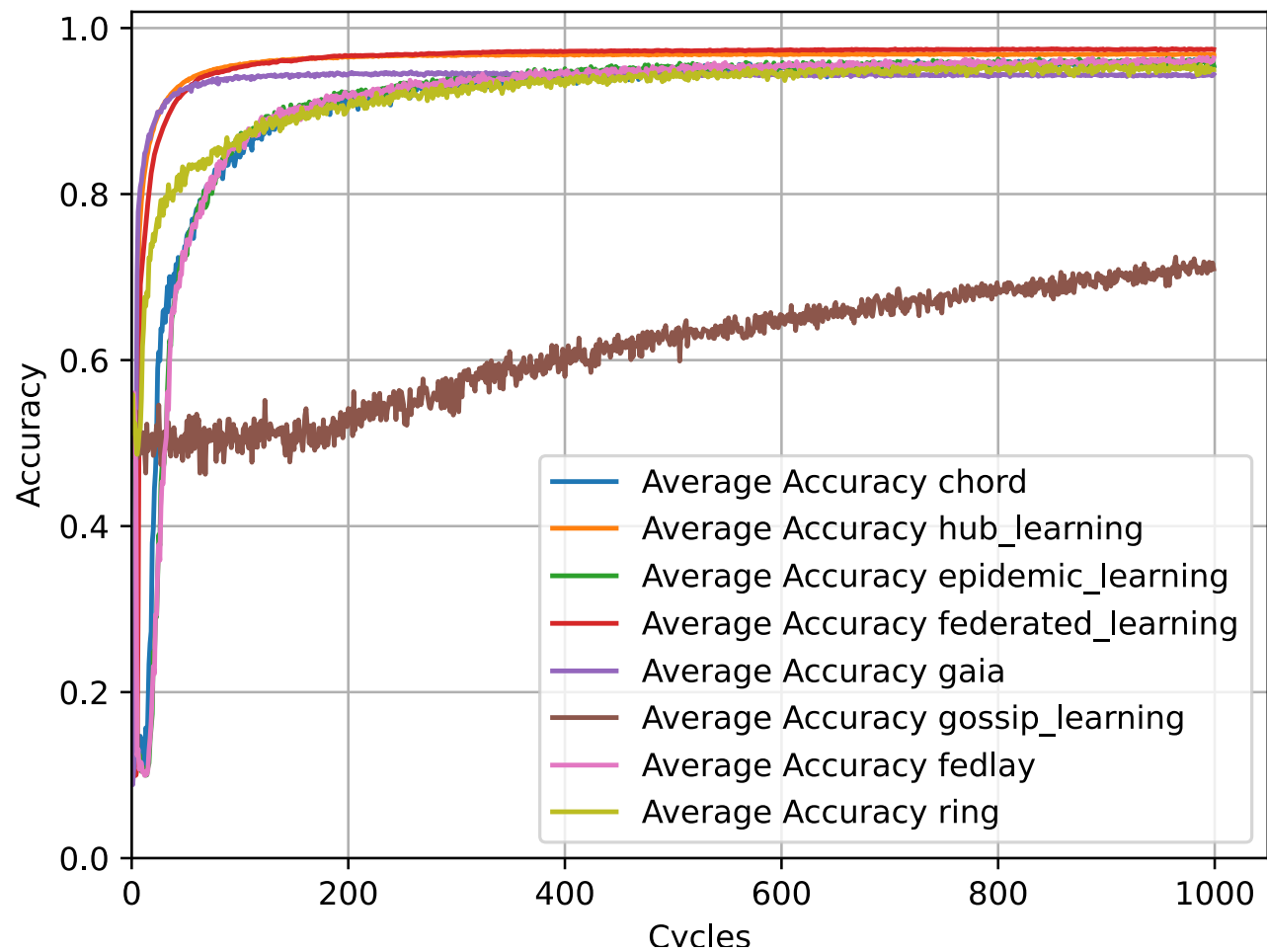
MNIST + LeNet5

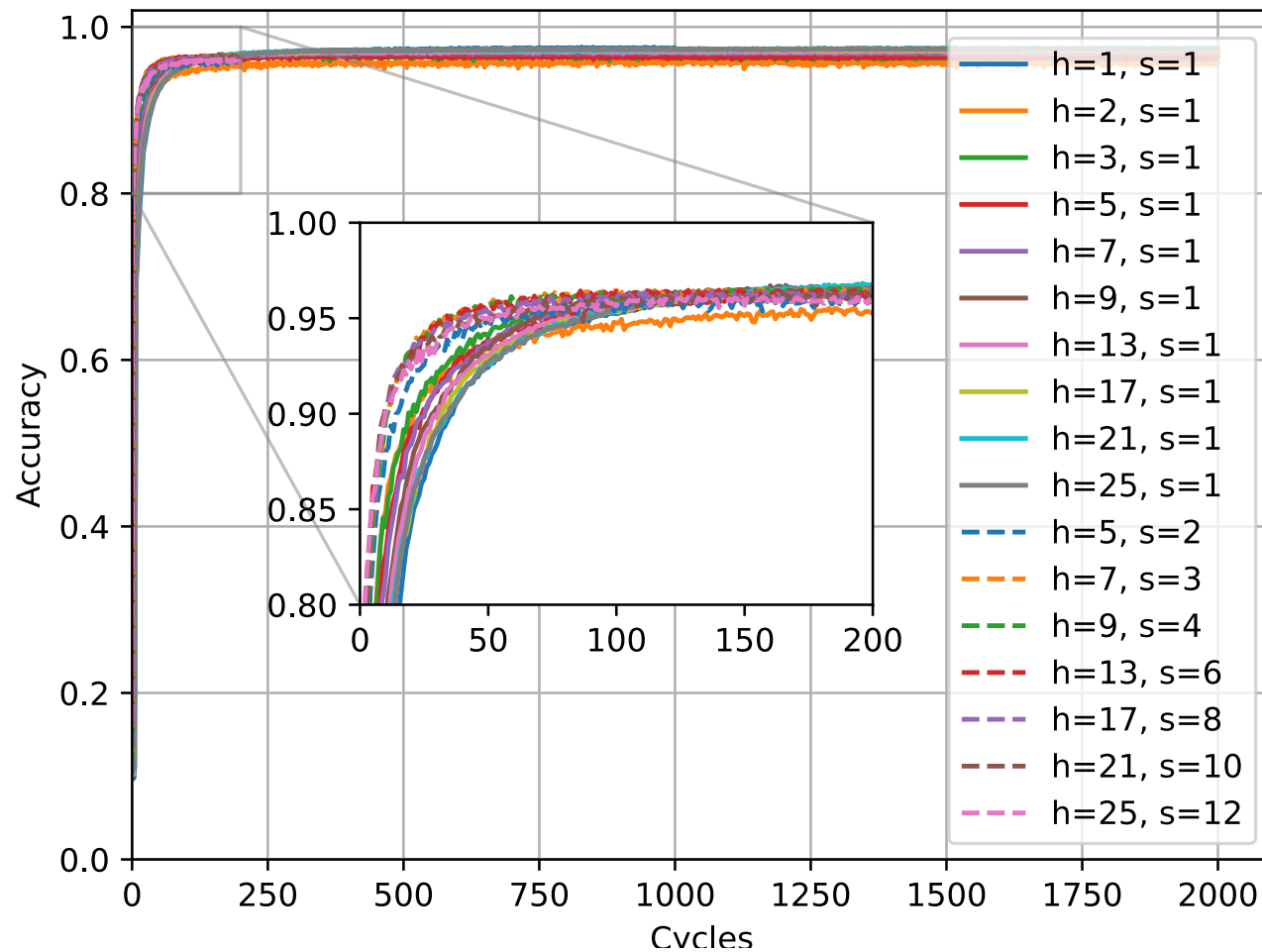


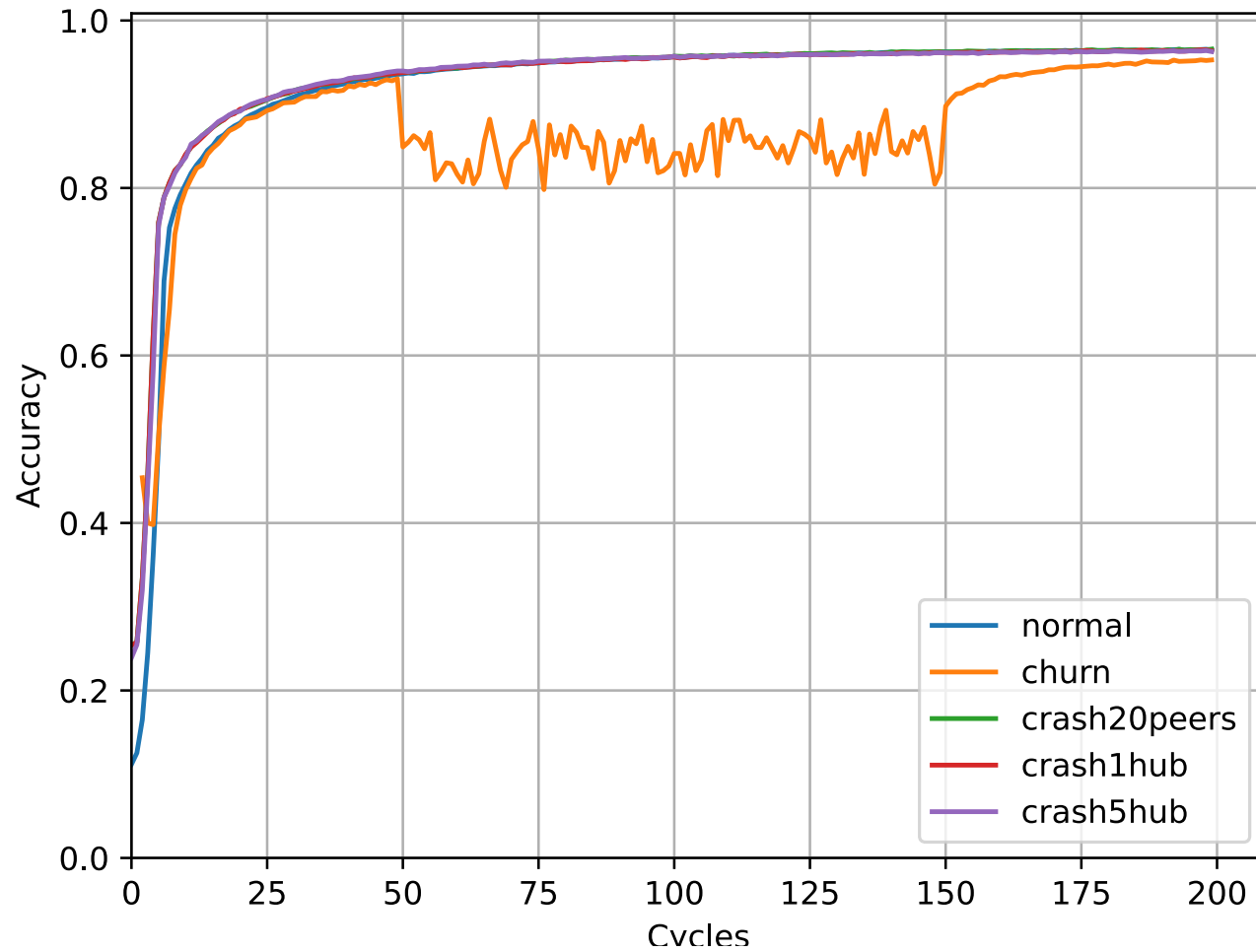
70k digits, 10 classes
CNN, 60k parameters

Context of experiments

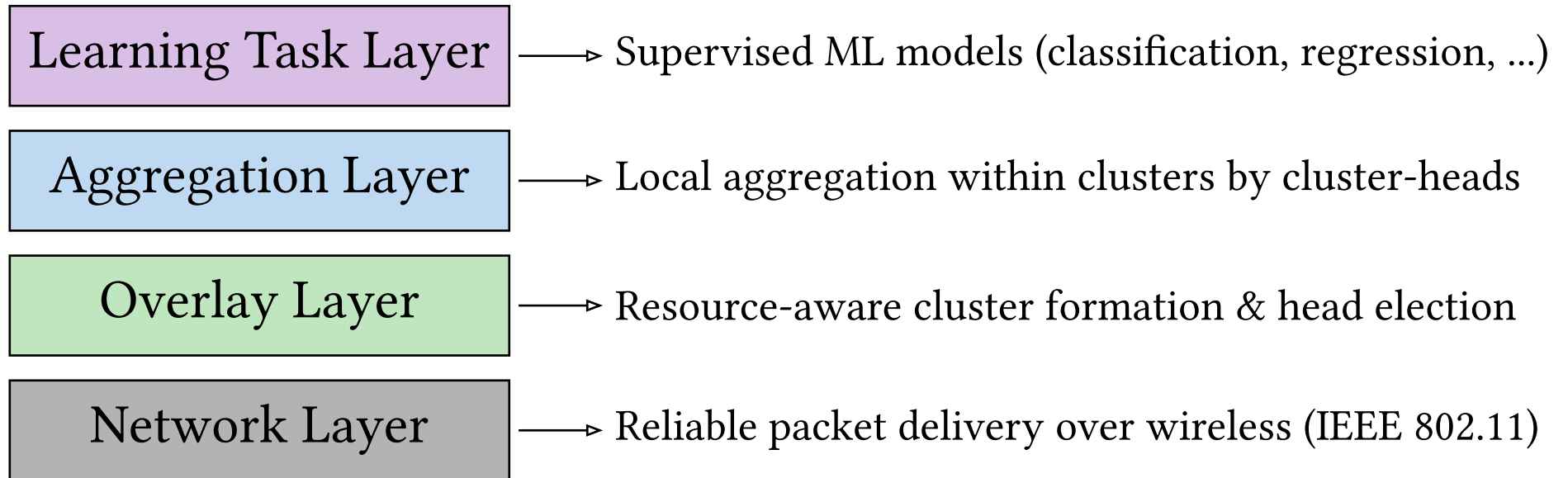
- **Peer-to-Peer:** On the PeerSim simulator.
 - Each generated network had 100 nodes, and the Hub-based topology had 5 hubs
 - In addition to the crash-free context, we ran the algorithm during a context of crash, a context of churn and a context of an attack on the hubs
- **Decentralized Learning:** On the gossip simulator
 - **ML models:** Logistic Regression & LeNet5
 - **Datasets:** Spambase and MNIST
 - Comparison with Federated Learning, Gossip Learning, Epidemic Learning, GAIA, Chord-based Learning, Fedlay







FLAIR architecture



FLAIR algorithm

FLAIR operates in **rounds** (clustering inspired by LEACH); a fraction $p \in (0, 1)$ of nodes serve as **cluster-heads** (CHs) each round, and the role **rotates** to balance the load.

A node eligible (not a CH during the last $1/p$ rounds) elects itself if $x < T(n)$, with $x \sim \text{Uniform}(0, 1)$ drawn via a **VRF**:

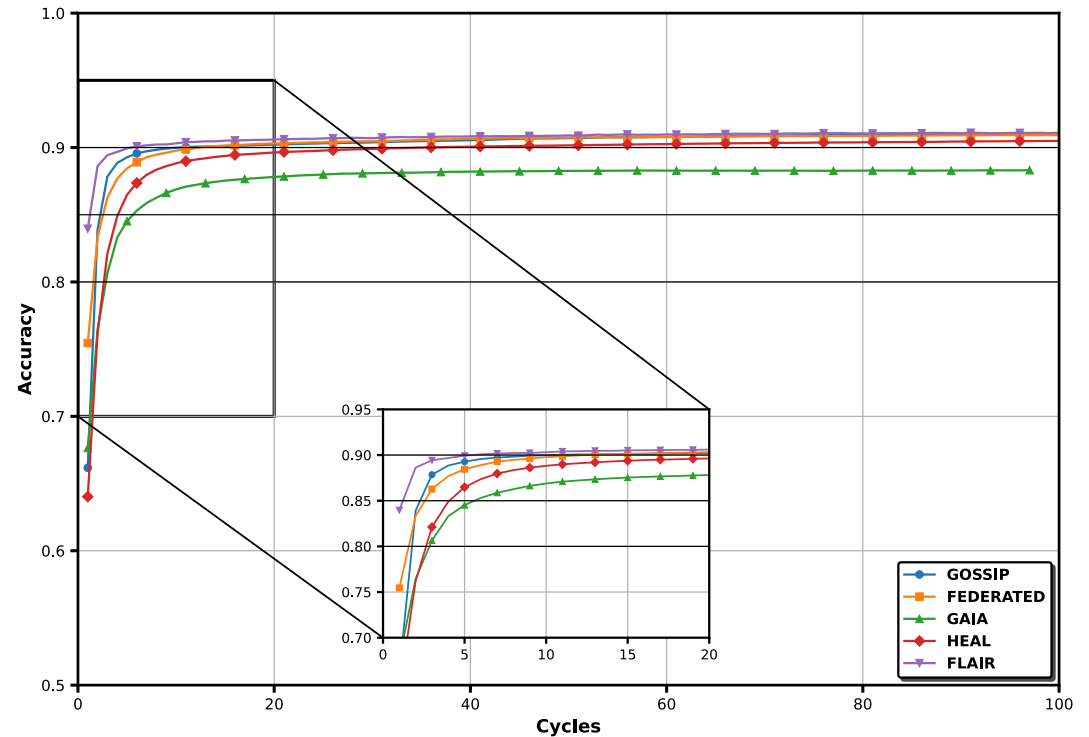
$$T(n) = p \cdot R_n / (1 - p \cdot (r \bmod 1/p))$$

with $R_n = \alpha \cdot \text{CPU}_n + \beta \cdot \text{RAM}_n + \gamma \cdot \text{GPU}_n + \delta \cdot \text{BW}_n$ (resource score; $\alpha + \beta + \gamma + \delta = 1$).

Nodes join the nearest CH; the CH **aggregates locally** within its cluster.

FLAIR simulation results

- Static network (100 nodes): **highest accuracy** ≈ 0.91 (C-FL / HEAL ≈ 0.90 , Gossip ≈ 0.88)
- Convergence up to **2.5x faster** than Gaia
- Resilient to **permanent / temporary / random crashes** (up to 90% nodes)
- Tested under **mobility** and in a **smart farming** scenario



Conclusion

- **Elevator**: a self-organising overlay that makes hubs emerge — provides a **structured overlay** for structured aggregation without any coordinator.
- **HEAL**: federated learning directly onto the structured overlay — recovers the efficiency of FL while staying decentralised.
- **Lift**: hardens hub election against colluding byzantines (up to 10%).
- **FLAIR**: validates the modular architecture in wireless networks (LEACH-style clustering).

Answer: structured aggregation is possible **without centralisation**, if the overlay layer is designed with that goal in mind.

Perspectives — short term

- **Capability-aware hub election** (heterogeneous nodes) and sensitivity to the initial topology.
- Optimise Elevator (message complexity, memory, simplicity).
- Address **data heterogeneity** in HEAL (personalisation, locally-weighted aggregation).
- A unified **Python simulator** for peer sampling + ML.

This work has already begun during a **3-month internship at NII (Tokyo)**.

Perspectives — medium & long term

- Better robustness at the Overlay layer (using)
- **Byzantine robustness in the learning layer** (poisoning & privacy attacks)
- Extend HEAL to **unsupervised and reinforcement learning**.
- **Vertical federated learning** (different features per participant).
- **Formal convergence guarantees** for HEAL.

Publications

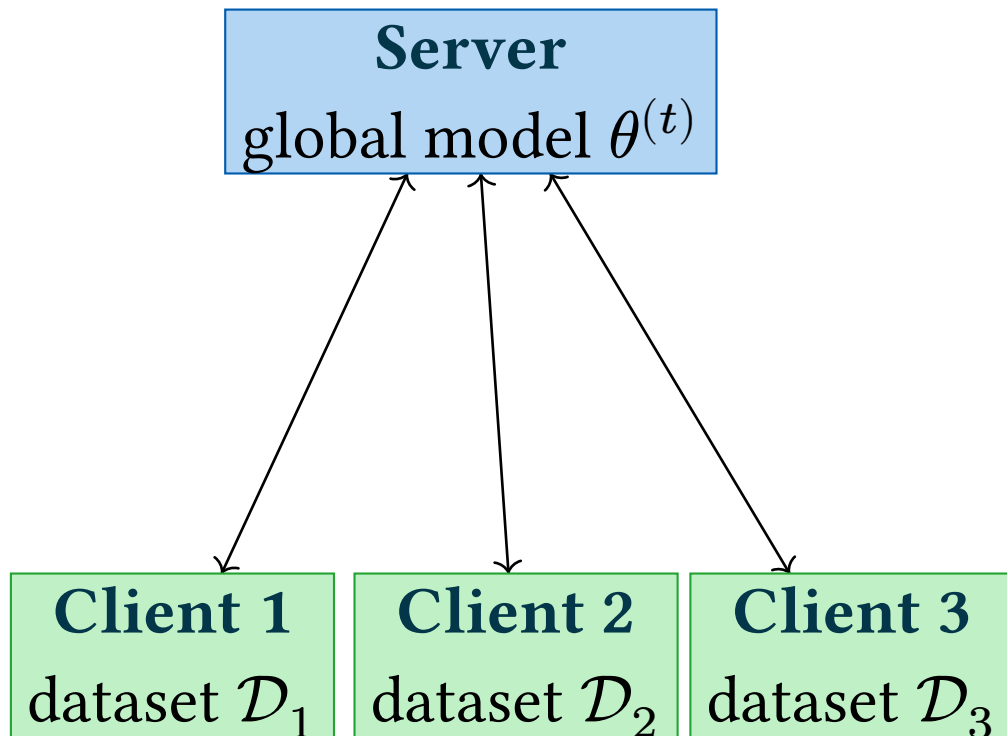
- [4] M. A. Legheraba, M. Potop-Butucaru, and S. Tixeuil, “Brief Announcement: A Self-* and Persistent Hub Sampling Service,” in *International Symposium on Stabilizing, Safety, and Security of Distributed Systems*, 2024, pp. 461–465.
- [5] M. A. Legheraba, M. Potop-Butucaru, S. Tixeuil, and S. Fdida, “Emergent Peer-to-Peer Multi-Hub Topology,” in *2024 22nd International Symposium on Network Computing and Applications (NCA)*, 2024, pp. 219–226.
- [6] M. A. Legheraba, N. Rachdi, M. Potop-Butucaru, and S. Tixeuil, “LIFT: Byzantine Resilient Hub-Sampling,” in *2025 Thirteenth International Symposium on Computing and Networking (CANDAR)*, 2025, pp. 1–9. — **Outstanding Paper Award**
- [7] M. A. Legheraba, S. Galkiewicz, M. Potop-Butucaru, and S. Tixeuil, “HEAL: Resilient and Self-* Hub-based Learning,” in *International Conference on Advanced Information Networking and Applications*, 2025, pp. 200–211.
- [8] I. Boutebicha, B. Zaghdoudi, M. A. Legheraba, and M. P. Butucaru, “FLAIR: Distributed Federated Learning with Dynamic Clustering,” in *Proceedings of the 14th International Conference on Networked Systems (NETYS 2026)*, in Lecture Notes in Computer Science. Agadir, Morocco: Springer, 2026.
- [9] M. A. Legheraba, M. Potop-Butucaru, S. Tixeuil, and S. Fdida, “Des noeuds anonymes hissés au rang de stars,” in *ALGOTEL 2025–27èmes Rencontres Francophones sur les Aspects Algorithmiques des Télécommunications*, 2025.
- [10] M. A. Legheraba, S. Galkiewicz, M. Potop-Butucaru, and S. Tixeuil, “Les étoiles montantes de l'apprentissage distribué,” in *ALGOTEL 2025–27èmes Rencontres Francophones sur les Aspects Algorithmiques des Télécommunications*, 2025.
- A journal extension (journal version of this work) has been **submitted to the IEEE/ACM Transactions on Networking**; we are awaiting the **final review** (currently in **minor revision**).

Bibliography

- [1] A.-L. Barabási, H. Jeong, Z. Néda, E. Ravasz, A. Schubert, and T. Vicsek, “Evolution of the social network of scientific collaborations,” *Physica A: Statistical mechanics and its applications*, vol. 311, no. 3–4, pp. 590–614, 2002.
- [2] A. Stavrou, D. Rubenstein, and S. Sahu, “A lightweight, robust p2p system to handle flash crowds,” in *10th IEEE International Conference on Network Protocols, 2002. Proceedings.*, 2002, pp. 226–235.
- [3] M. Jelasity, S. Voulgaris, R. Guerraoui, A.-M. Kermarrec, and M. Van Steen, “Gossip-based peer sampling,” *ACM Transactions on Computer Systems (TOCS)*, vol. 25, no. 3, p. 8–es, 2007.
- [4] M. A. Legheraba, M. Potop-Butucaru, and S. Tixeuil, “Brief Announcement: A Self-* and Persistent Hub Sampling Service,” in *International Symposium on Stabilizing, Safety, and Security of Distributed Systems*, 2024, pp. 461–465.
- [5] M. A. Legheraba, M. Potop-Butucaru, S. Tixeuil, and S. Fdida, “Emergent Peer-to-Peer Multi-Hub Topology,” in *2024 22nd International Symposium on Network Computing and Applications (NCA)*, 2024, pp. 219–226.
- [6] M. A. Legheraba, N. Rachdi, M. Potop-Butucaru, and S. Tixeuil, “LIFT: Byzantine Resilient Hub-Sampling,” in *2025 Thirteenth International Symposium on Computing and Networking (CANDAR)*, 2025, pp. 1–9.
- [7] M. A. Legheraba, S. Galkiewicz, M. Potop-Butucaru, and S. Tixeuil, “HEAL: Resilient and Self-* Hub-based Learning,” in *International Conference on Advanced Information Networking and Applications*, 2025, pp. 200–211.

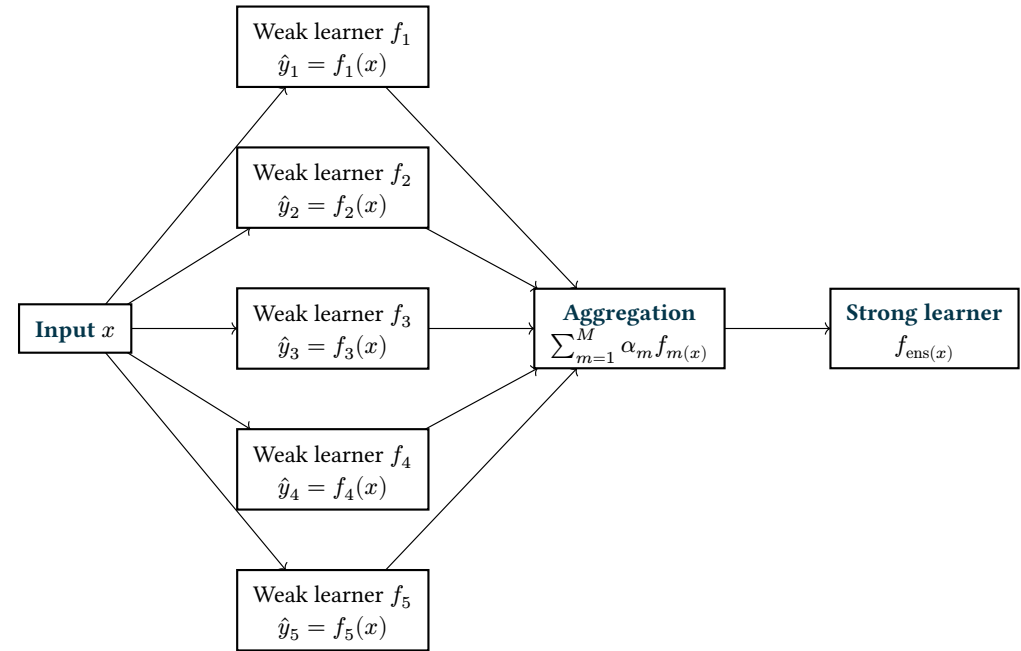
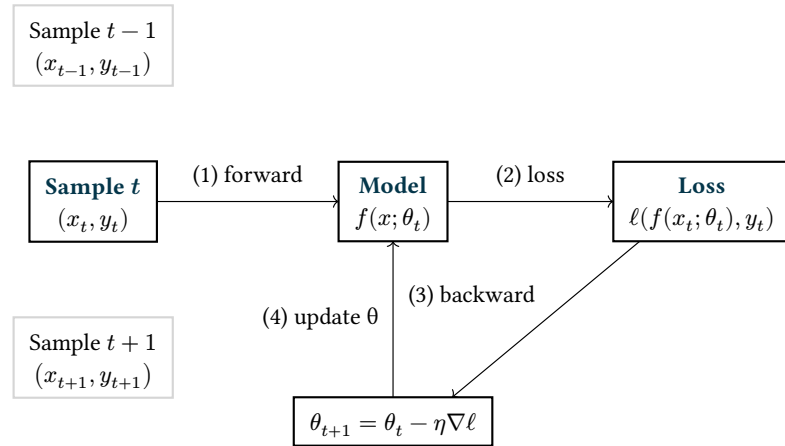
- [8] I. Boutebicha, B. Zaghdoudi, M. A. Legheraba, and M. P. Butucaru, “FLAIR: Distributed Federated Learning with Dynamic Clustering,” in *Proceedings of the 14th International Conference on Networked Systems (NETYS 2026)*, in Lecture Notes in Computer Science. Agadir, Morocco: Springer, 2026.
- [9] M. A. Legheraba, M. Potop-Butucaru, S. Tixeuil, and S. Fdida, “Des noeuds anonymes hissés au rang de stars,” in *ALGOTEL 2025–27èmes Rencontres Francophones sur les Aspects Algorithmiques des Télécommunications*, 2025.
- [10] M. A. Legheraba, S. Galkiewicz, M. Potop-Butucaru, and S. Tixeuil, “Les étoiles montantes de l'apprentissage distribué,” in *ALGOTEL 2025–27èmes Rencontres Francophones sur les Aspects Algorithmiques des Télécommunications*, 2025.
- [11] A. Montresor and M. Jelasity, “PeerSim: A Scalable P2P Simulator,” in *Proc. of the 9th Int. Conference on Peer-to-Peer (P2P'09)*, Seattle, WA, Sep. 2009, pp. 99–100.

Appendix



Federated Learning (FedAvg)

```
1 for  $t = 0$  to  $T - 1$  do
2   server broadcasts  $\theta^{(t)}$ 
   for each client  $i$  in parallel:
3      $\theta_i \leftarrow$  local training (lr  $\eta$ ,  $E$ 
       steps)
4     server aggregates:  $\theta^{(t+1)} \leftarrow$ 
        $\sum w_i \theta_i$ 
5 end for
```



HEAL Learning: The Hub Algorithm

```
duration to wait for model: delta_time
list of all hubs: hubs_list
1 nb_hubs  $\leftarrow$  hubs_list.size()
2 loop
3   models  $\leftarrow$  {}
4   time  $\leftarrow$  time.now()
5   backwards_list  $\leftarrow$  {}
6   while time.now() < time + delta_time do
7     (peer_model, peer)  $\leftarrow$  receive()
8     models.append(peer_model)
9     backwards_list.append(peer)
10  average_model  $\leftarrow$  average(models)
11  send(hubs_list, average_model)
12  models_hubs  $\leftarrow$  {}
13  models_hubs.append(average_model)
14  nb_receive  $\leftarrow$  0
15  while nb_receive < nb_hubs - 1 do
16    hub_model  $\leftarrow$  receive()
17    nb_receive  $\leftarrow$  nb_receive + 1
18    models_hubs.append(hub_model)
19  global_model  $\leftarrow$  average(models_hubs)
20  send(backwards_list, global_model)
```

HEAL Learning: The Client Algorithm

```
duration to wait for model: delta_time
The ML model, initialized at random: model
The local data: data
list of all hubs: hubs_list
number of hubs to send the model: s
1 loop
2   model  $\leftarrow$  trainModel(model, data)
3   hubs  $\leftarrow$  chooseRandom(hubs_list, s)
4   for hub in hubs do
5     | send(hub, model)
6   hubs_models  $\leftarrow$  {}
7   for hub in hubs do
8     | model_hub  $\leftarrow$  receive()
9     | hubs_models.append(model_hub)
10  model  $\leftarrow$  average(hubs_models)
```

Elevator Algorithm (background thread)

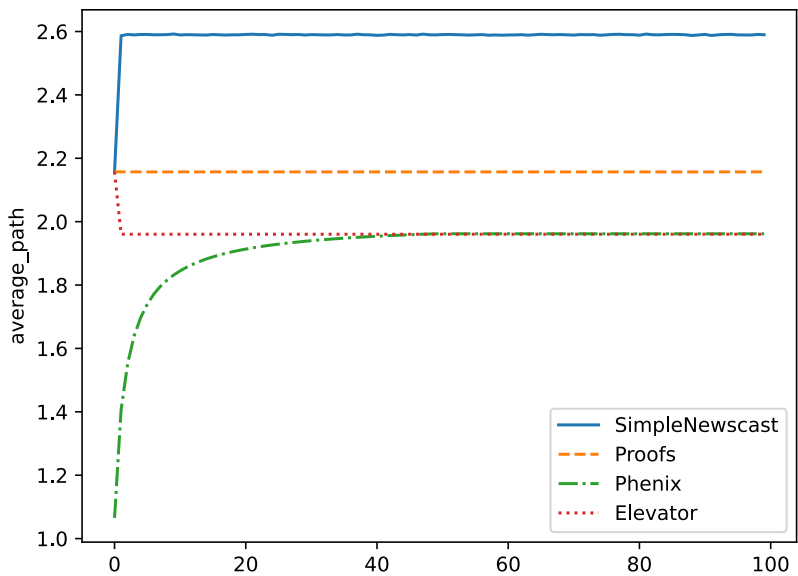
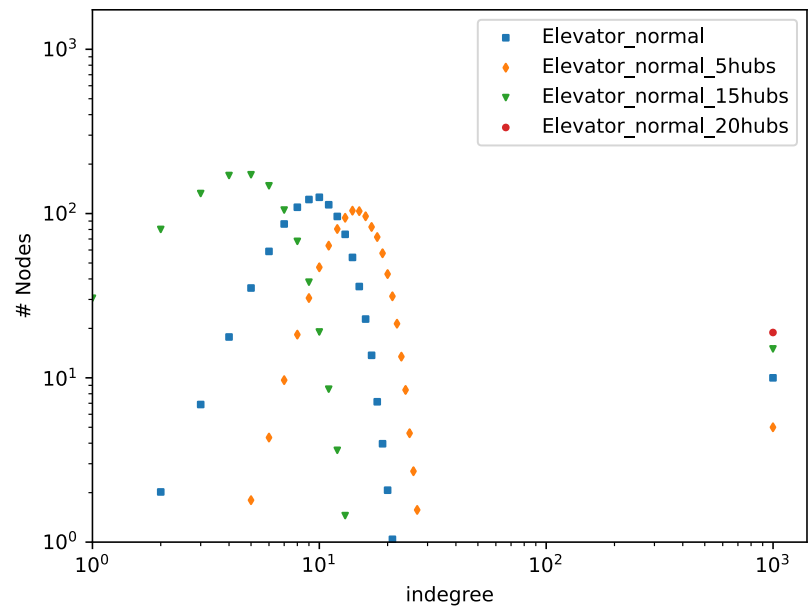
```
1 loop
2   request, peer  $\leftarrow$  receive()
3   if request == CACHE_REQUEST then
4     send(cache, peer)
5     backward_peers.add(peer)
6   if request == BACKWARD_REQUEST then
7     send(backward_peers, peer)
```

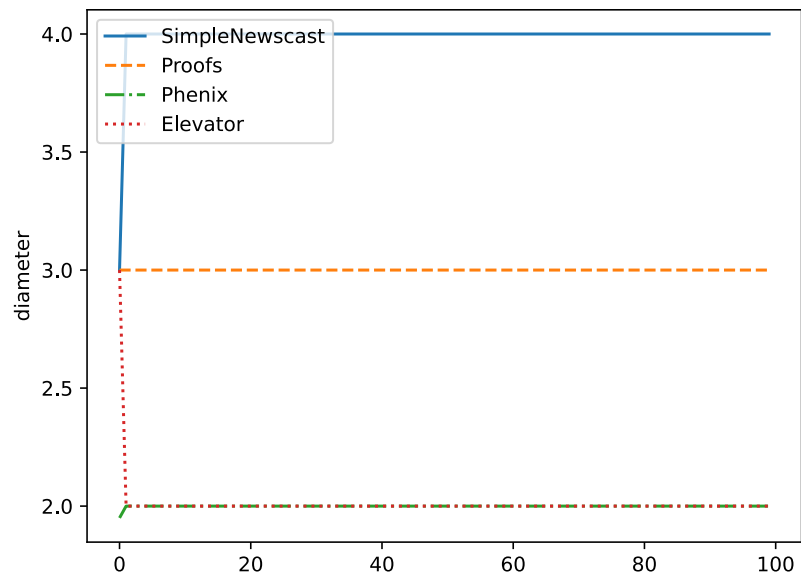
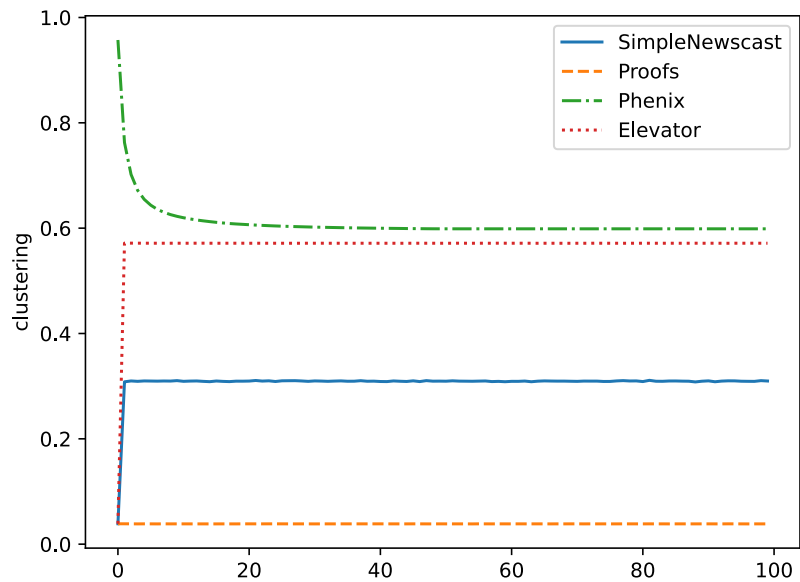
- Simulations done with the Java PeerSim^{*} simulator (modified),
with the cycle based mode
- Comparaisons against 3 peer sampling algorithms : Newscast, Proofs and Phenix (Power-law)

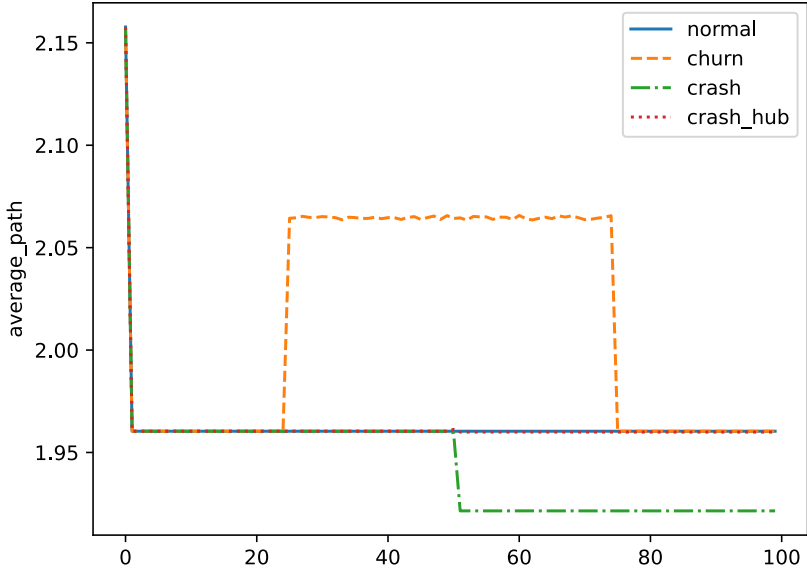
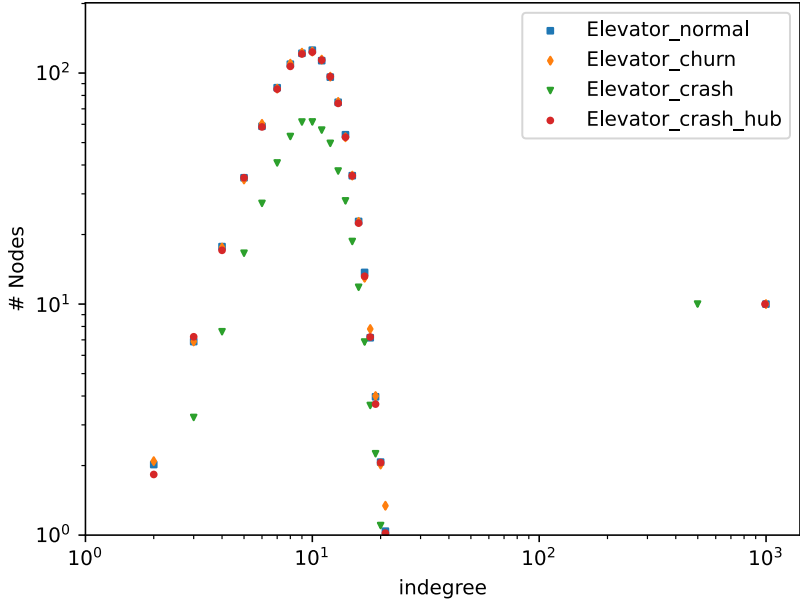
	Value
Network size	1000
Number of cycles for each simulation	1000
Number of times each simulation was run (with different seed)	100
c parameter (cache size)	20
h parameter (number of hubs)	10

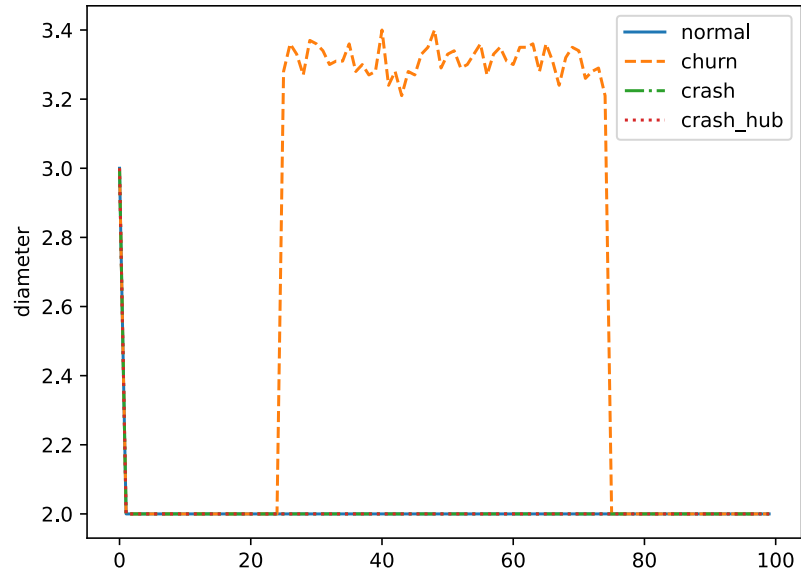
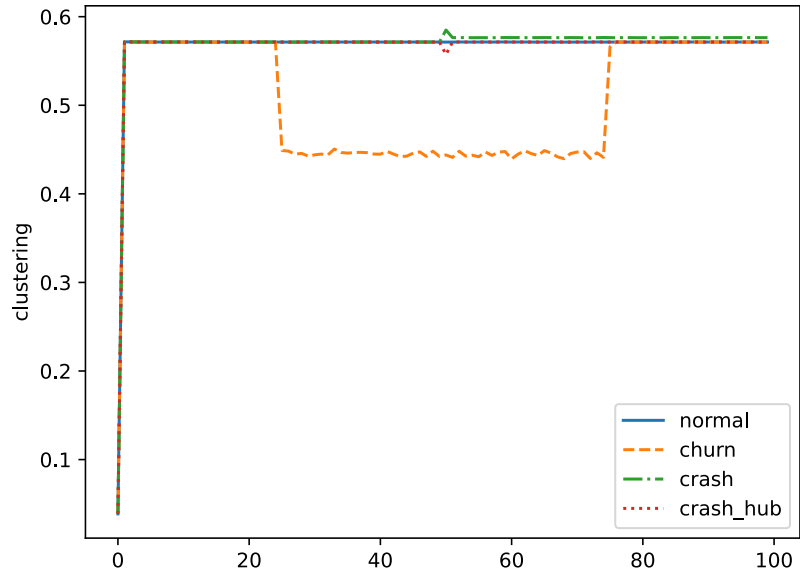
- All simulations were run on 16 vCPU, using 64G of memory.
- Code is available at <https://gitlab.lip6.fr/legheraba/elevator>

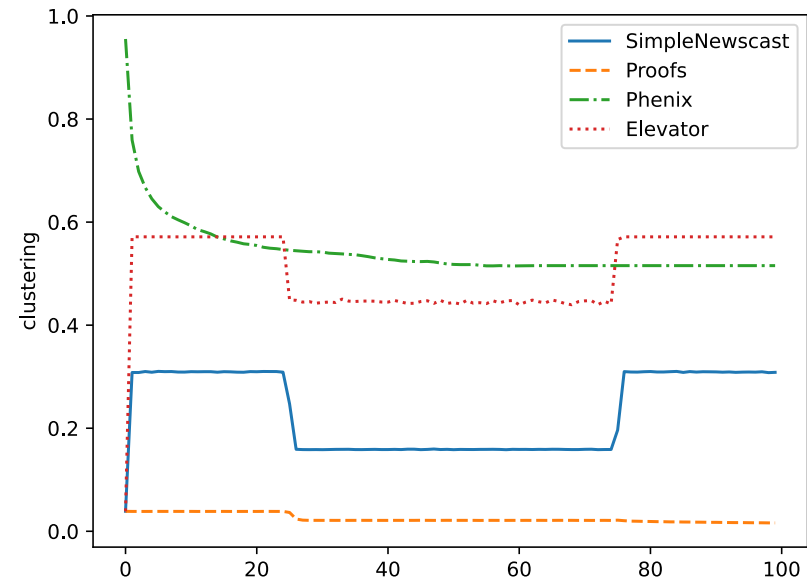
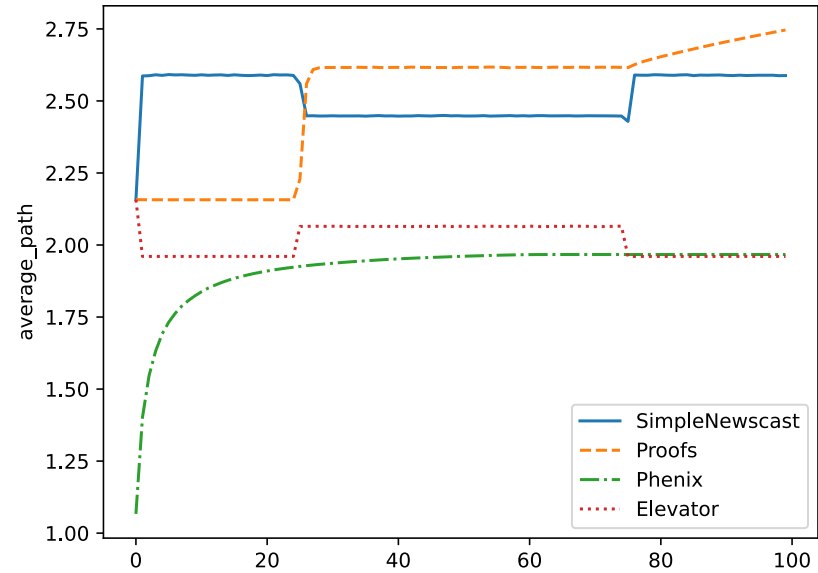
^{*}[11] A. Montresor and M. Jelasity, “PeerSim: A Scalable P2P Simulator,” in *Proc. of the 9th Int. Conference on Peer-to-Peer (P2P'09)*, Seattle, WA, Sep. 2009, pp. 99–100.

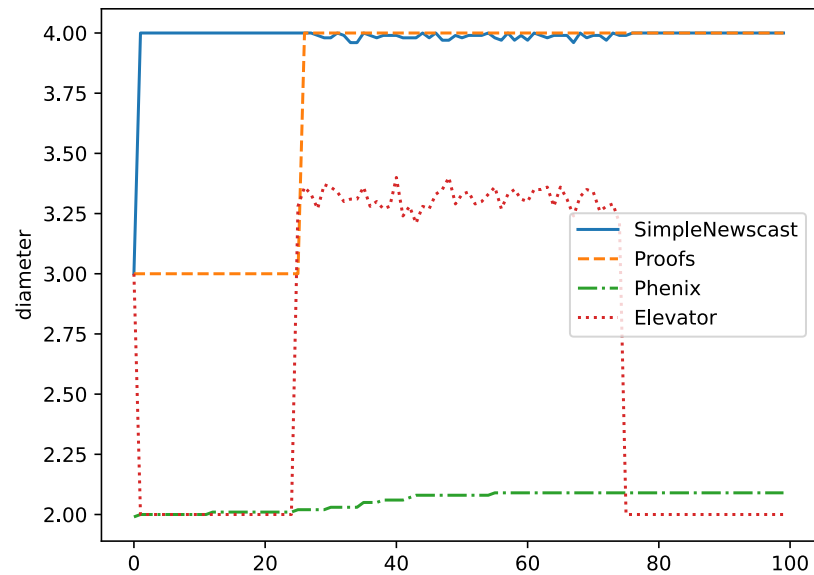












Federated Learning algorithm

Federated Learning (for the Server)

Number of iterations: **T**

Number of clients: **K**

Fraction of clients per iteration: **C**

```

1 for each  $t = 1 \dots T$  do
2    $m \leftarrow \max(C \cdot K, 1)$ 
3    $S_t \leftarrow \text{selectRandomSubset}(m, \text{clients})$ 
4   for each  $k \in S_t$ 
5      $\text{send}(k, w_t)$ 
6      $w_{t+1}^k \leftarrow \text{receive}(k)$ 
7    $m_t \leftarrow \sum_{k \in S_t} n_k$ 
8    $w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} w_{t+1}^k$ 

```

Federated Learning (for a Client)

Local dataset: D_k

Number of local epochs: **E**

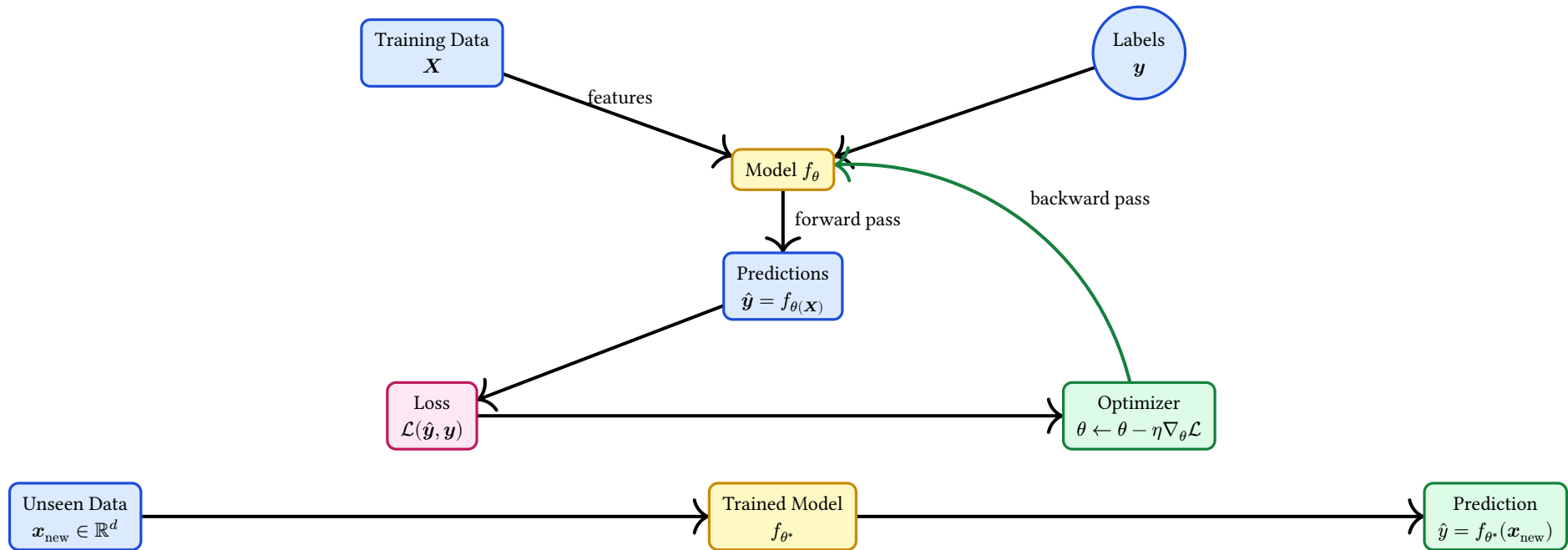
Learning rate: η

```

1  $\text{receive}(w_t)$ 
2 for each  $\text{epoch} = 1 \dots E$  do
3   for  $\text{batch } b \subset D_k$  do
4      $w_t \leftarrow w_t - \eta \cdot \nabla L(w_t; b)$ 
5  $\text{send}(w_t^k)$ 

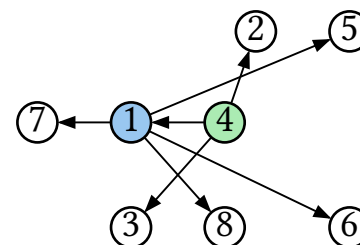
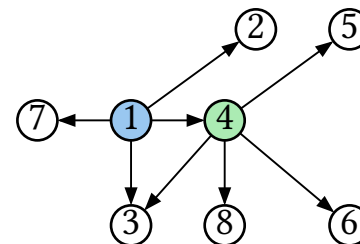
```

```
1 loop
2   if node_is_hub do
3     while time.now() < time_start + delta_time do
4       | models_from_nodes.append(receiveModelFromNode())
5       aggregated_model ← average(models_from_nodes)
6       sendAll(hubs_list, aggregated_model)
7       global_model ← average(models_from_other_hubs)
8       send(nodes_list, global_model)
9   else
10    hubs ← chooseRandom(hubs_list, number_hubs_to_send_model)
11    model ← trainModel(model, data)
12    sendAll(hubs, model)
13    for hub in hubs do
14      | global_models.append(receiveGlobalModelFromHub())
15    model ← average(global_models)
```



PROOFS Algorithm (active thread)

```
initial peer list: cache
cache size: c
shuffle length: l
node address: p
1 loop
2   wait( $\Delta$ )
3   subset  $\leftarrow$  selectRandomSubset(cache, l)
4   q  $\leftarrow$  selectRandom(subset)
5   subset.remove(q)
6   subset.add(p)
7   send(q, subset)
8   subsetq  $\leftarrow$  receive(q)
9   subsetq.remove(p)
10  subsetq.removeAll(cache)
11  cache  $\leftarrow$  subsetq
```



Before and after a shuffling operation. Node 1 sends addresses {itself, 2, 3} to node 4. Node 4 sends back {5,6, 8}.

- The objective is to obtain an overlay network with **h** defined hubs, with **h** a parameter of the algorithm, and each hub is connected to all the nodes in the networks. The application running on top will be able to take advantage of this overlay to speed up message transmission in the network.
- **Preferential Attachment**: Drawing from the concept pioneered by Barabási and Albert^{*}, preferential attachment dictates that new connections in the network are established preferentially with nodes possessing a higher number of existing connections.
- **Random Attachment**: Inspired by gossip-based peer sampling algorithms (**), random attachment ensures that nodes maintain connections with a representative and diverse subset of the network.

^{*}[1] A.-L. Barabási, H. Jeong, Z. Néda, E. Ravasz, A. Schubert, and T. Vicsek, “Evolution of the social network of scientific collaborations,” *Physica A: Statistical mechanics and its applications*, vol. 311, no. 3–4, pp. 590–614, 2002.

^{*}[2] A. Stavrou, D. Rubenstein, and S. Sahu, “A lightweight, robust p2p system to handle flash crowds,” in *10th IEEE International Conference on Network Protocols, 2002. Proceedings.*, 2002, pp. 226–235.

^{*}[3] M. Jelasity, S. Voulgaris, R. Guerraoui, A.-M. Kermarrec, and M. Van Steen, “Gossip-based peer sampling,” *ACM Transactions on Computer Systems (TOCS)*, vol. 25, no. 3, p. 8–es, 2007.

Elevator Algorithm (active thread)

```
initial peer list: cache
cache size: c
desired number of hubs: h
initial backward list: backward_peers (empty)
1 loop
2   wait( $\Delta$ )
3   frequency_map  $\leftarrow \{\}$ 
4   for peer in cache do
5     | peer_cache  $\leftarrow$  send(CACHE_REQUEST, peer)
6     | frequency_map  $\leftarrow$  frequency_map  $\cup$  peer_cache
7   preferred  $\leftarrow$  frequency_map.sort().select(h)
8   preferred_backward  $\leftarrow \{\}$ 
9   for peer in preferred do
10    | peer_backward_peer  $\leftarrow$  send(BACKWARD_REQUEST,
11    | peer)
12    | preferred_backward.add(peer_backward_peer)
13   cache  $\leftarrow \{\}$ 
14   cache  $\leftarrow$  preferred  $\cup$  preferred_backward
14   backward_peers  $\leftarrow \{\}$ 
```

Elevator Algorithm (background thread)

```
1 loop
2   request, peer  $\leftarrow$  receive()
3   if request == CACHE_REQUEST then
4     | send(cache, peer)
5     | backward_peers.add(peer)
6   if request == BACKWARD_REQUEST
7   then
8     | send(randomValue(backward_peers),
9     | peer)
```



Peer sampling

Peer discovery

Data request

**Membership
management**

**Topology
management**

**Information
dissemination**